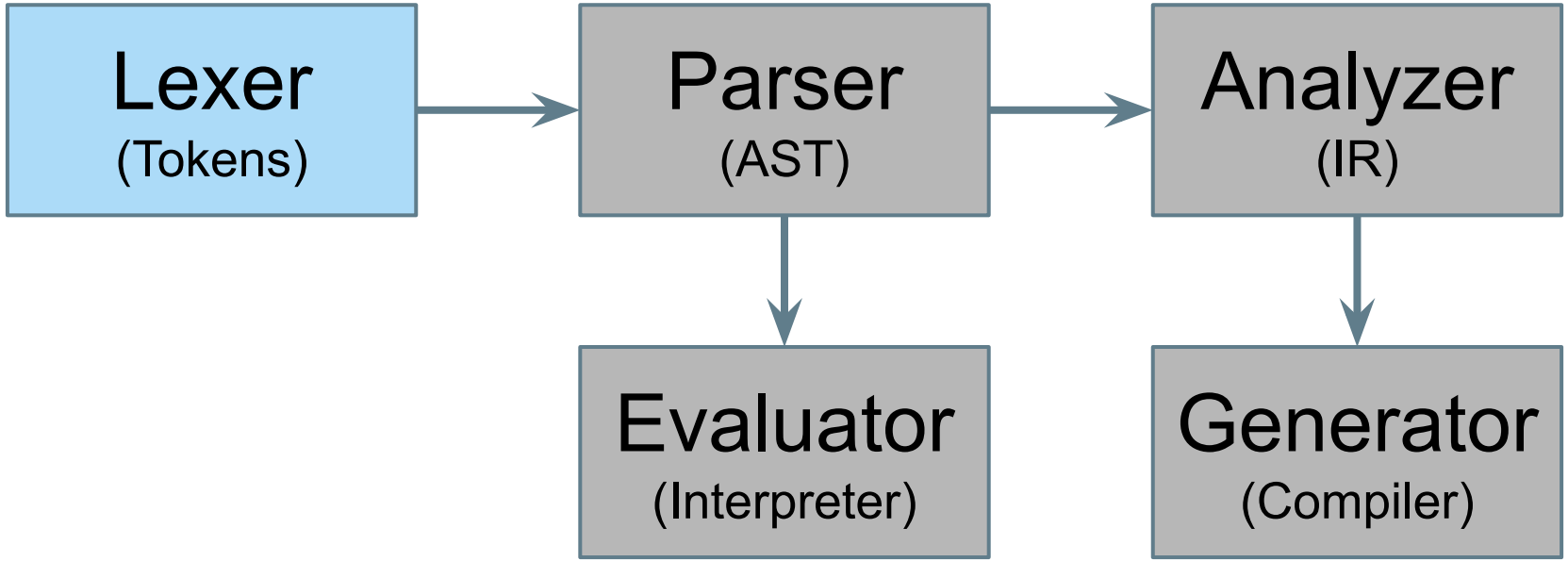
Module 1: Lexing

Lexer Introduction

It's time to begin, isn't it?

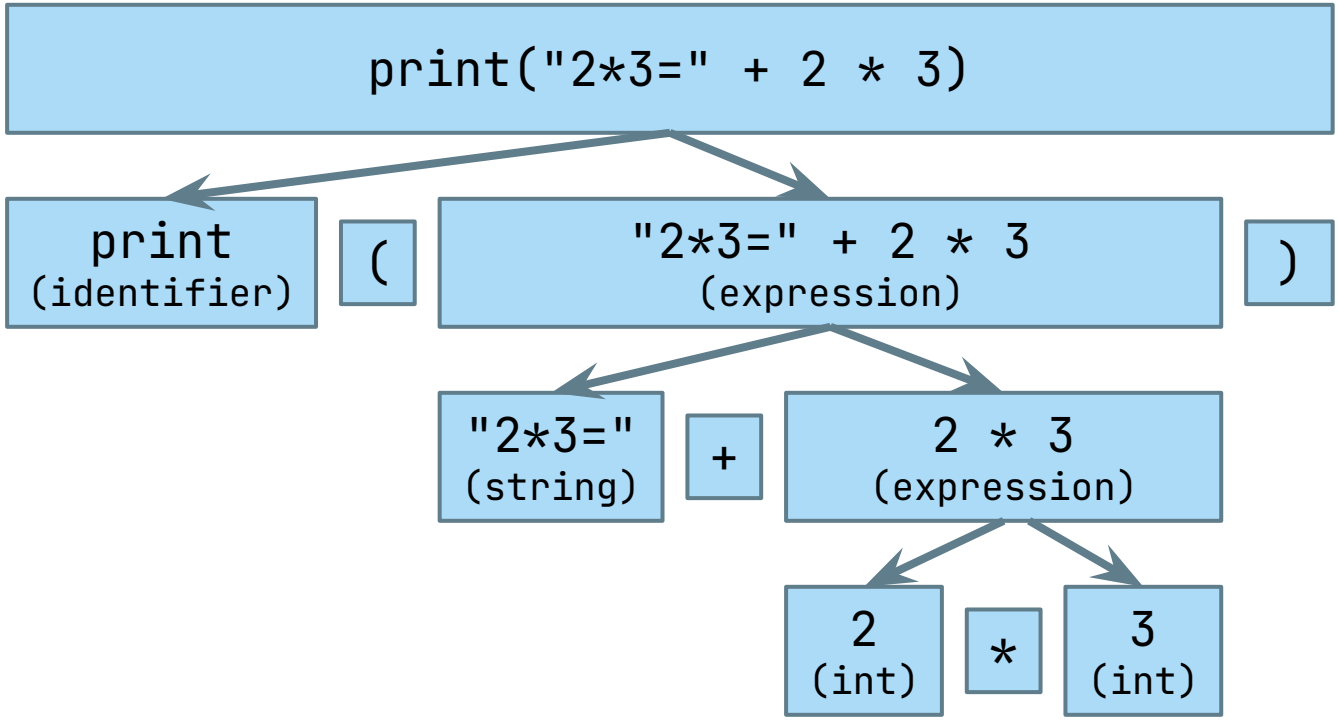
The Big Idea (!)


Lexer
(Tokens)
Parser
(AST)
Analyzer
(IR)
Evaluator
(Interpreter)
Generator
(Compiler)

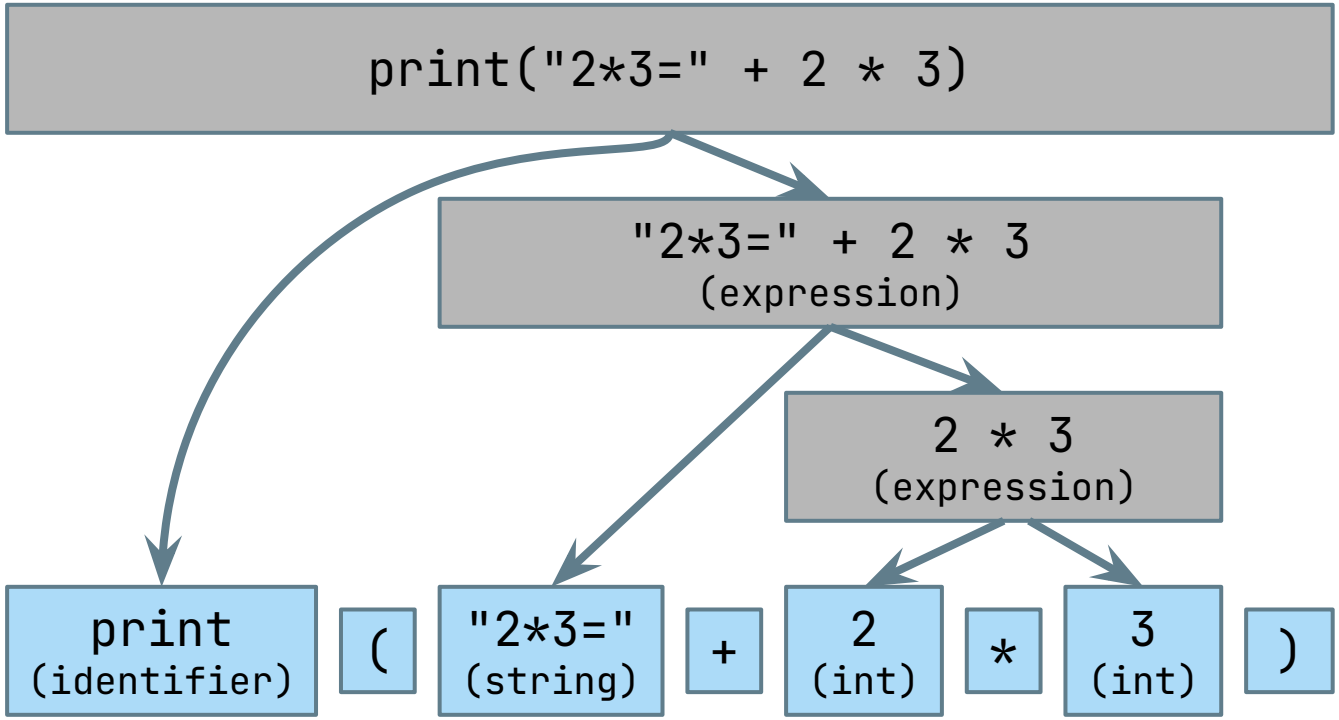
1.

Lexing

So perplexing!



Parser Goal: Structured Input (AST)



Lexer Goal: Grouped Input (Tokens)

Lexing

Definitions

Lexing (or Scanning) is the process of grouping characters into **tokens**.

- Tokens are the “building blocks” of grammars that define a language.
  - Parsers care about tokens, not the individual string characters.
- The lexer is just a function of `String` to `List<Token>`.

Grammars

Grammars formally define which input strings are considered part of a language.

- Regex effectively defines a grammar for strings (and has it’s own grammar too!).
- However, real grammars are more powerful:
  - Grammars define named **rules** which can be referenced by other rules as well.
  - Rules can be <u>recursive</u>, allowing far more complex languages than regex.
  - More on this next lecture…

```
//"standard" BNF

number ::= integer | decimal
integer ::= digit+
decimal ::= digit+ dot digit+

digit ::= [0-9]
dot ::= '.'
```

```
//"fancy" EBNF

number ::= [0-9]+ ('.' [0-9]+)?

//equivalent, but clearer (probably)
//and matches code logic better.
```

EBNF (Extended Brackus-Naur Form) Grammars

Grammars can be ambiguous!

Grammars are intended to accept/reject…

…but in practice are also used to *parse!*

Ambiguity

Definitions

An **ambiguous** grammar has multiple valid **derivations** to accept an input.

- `int ::= [0-9]+` is ambiguous!
  - 123 can be `[123]`, `[1, 23]`, `[12, 3]`, or `[1, 2, 3]`.
- Other examples:
  - `1.0`: decimal, integer + operator
  - `if(true)`: if stmt, function call
  - C++ `A*B`: mult, var declaration

Resolutions

Ambiguity must be eliminated to leave only one valid derivation per the specification.

- Style: Rules can be rewritten to avoid ambiguity, often matching impl behavior.
- Maximal Munch: Prioritize the longest valid token that can be created from input (lexing).
- Precedence: Use rule order to indicate the priority of alternative derivations (parsing).
- “Committing”: Specify when a rule is expected to accept once partially matched.
  - Allows errors to be generated eagerly.

2.

Lexer Assignment

drumroll, please…

Module 0: Introduction

Course Overview

COP-4020: Programming Language Concepts

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals